INSTITUTO FEDERAL
Norte de Minas Gerais
Campus Januária

Deploy de Aplicações Modernas

Gabriel Fernandes dos Santos



Monolito + Bare Metal

MONOLITO

- **Unidade Única:** Aplicação desenvolvida, empacotada e implantada como um único bloco de código executável.
- **Forte Integração:** Componentes internos, lógica de negócio e banco de dados altamente acoplados.

BARE METAL

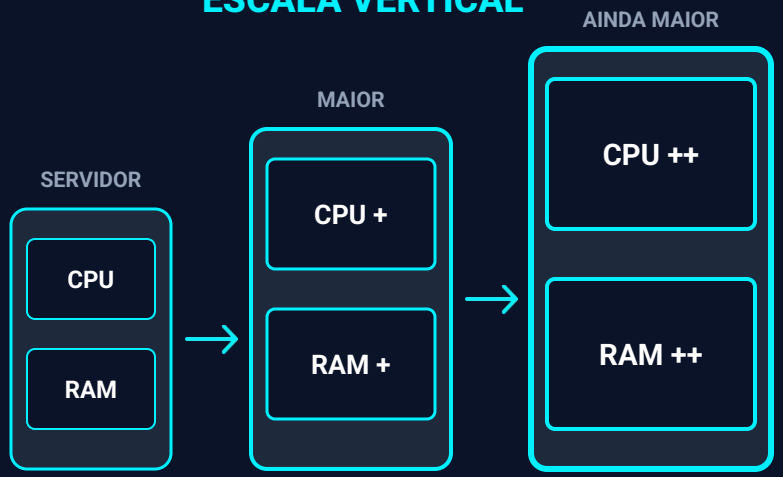
- **Servidor Físico:** Sem camadas de virtualização intermediando o acesso aos recursos.
- **Acesso Direto:** Desempenho máximo com CPU, RAM e armazenamento entregues diretamente ao sistema operacional.



LIMITAÇÕES

- **Limite Físico:** A escalabilidade vertical encontra um teto físico intransponível de hardware.
- **Alto Custo:** Upgrades de servidores de grande porte possuem custos exponenciais de aquisição e manutenção.
- **Recursos Ociosos:** Capacidade máxima dimensionada para picos gera desperdício em momentos de baixa demanda.

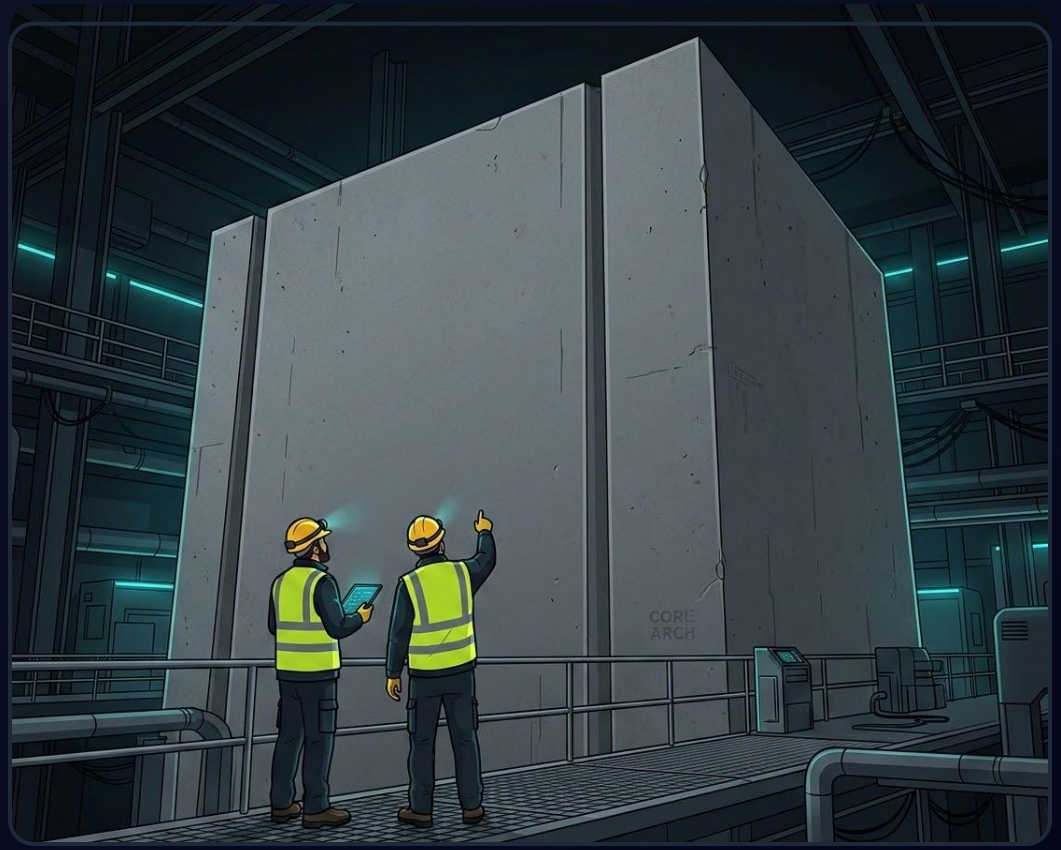
ESCALA VERTICAL



Nesse modelo, aumentar a capacidade do sistema exige a substituição ou expansão de **todo o hardware** de forma centralizada.

Monolito: uma estrutura rígida

- **Uma única estrutura:** Todo o sistema é construído e implantado como um bloco de pedra indivisível.
- **Componentes integrados:** Lógica, dados e interface fortemente conectados, onde qualquer alteração afeta o todo.
- **Difícil de escalar:** Não é possível escalar partes de forma independente, exigindo a replicação de toda a estrutura.



Microsserviços + Cloud Services

MICROSSERVIÇOS

- **Serviços independentes:** Funcionalidades desacopladas e autônomas.
- **Responsabilidades específicas:** Cada serviço foca em um único domínio de negócio.
- **Comunicação por APIs:** Troca de dados via APIs REST ou mensageria leve.
- **Escalabilidade independente:** Dimensione apenas o que demanda recursos.

CLOUD SERVICES

- **Recursos sob demanda:** Infraestrutura sob medida alocada dinamicamente.
- **Escalabilidade horizontal:** Adicione mais instâncias em vez de máquinas maiores.
- **Elasticidade natural:** Adaptação automática às variações de carga de acessos.
- **Menor custo físico:** Redução drástica na necessidade de data centers locais.

ARQUITETURA DE SERVIÇOS



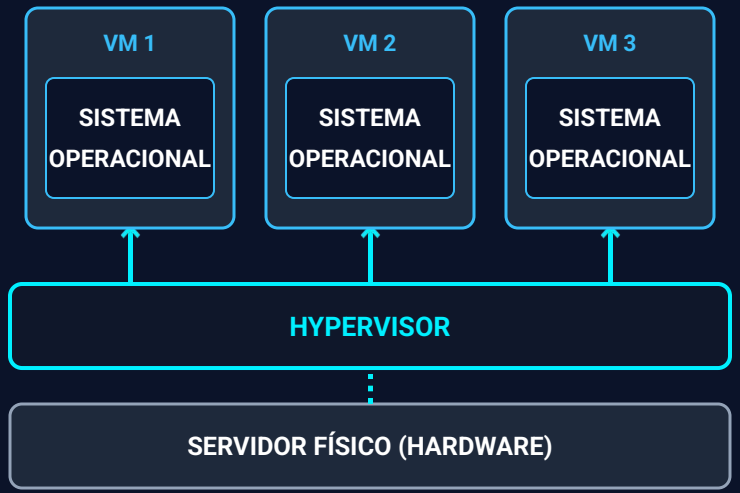


Virtualização

Conceito e Impacto

- **Hypervisor:** Camada de software intermediária que gerencia e distribui os recursos físicos do servidor.
- **Máquinas Virtuais (VMs):** Emulações independentes de hardware rodando sobre o mesmo servidor físico.
- **Sistemas Operacionais:** Cada VM possui e executa seu próprio SO completo e isolado.
- **Isolamento:** Segurança e estabilidade garantidas, falhas em uma VM não afetam as demais.
- **Alto Consumo:** Duplicação de SOs gera maior sobrecarga e consumo de recursos de hardware.

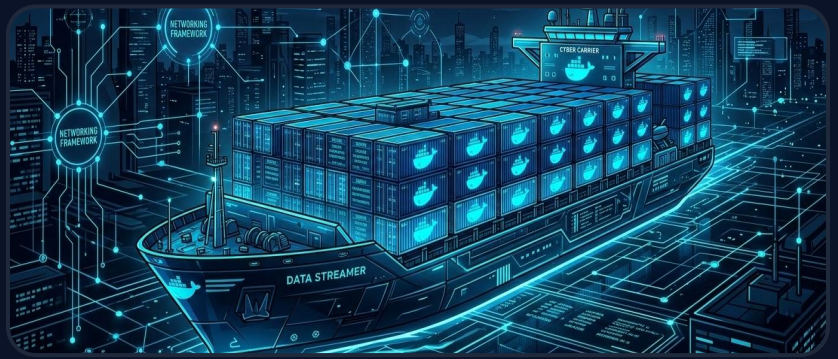
Arquitetura de Virtualização



Containerização e Docker

CONTAINERS

- **Isolam aplicações:** Processos rodam em ambientes totalmente isolados.
- **Compartilham o kernel:** Utilizam o sistema operacional do host.
- **Mais leves:** Dispensam a necessidade de carregar um SO completo.
- **Inicialização rápida:** Inicializam em poucos segundos.
- **Fácil replicação:** Facilidade de criar cópias e escalar de forma consistente.



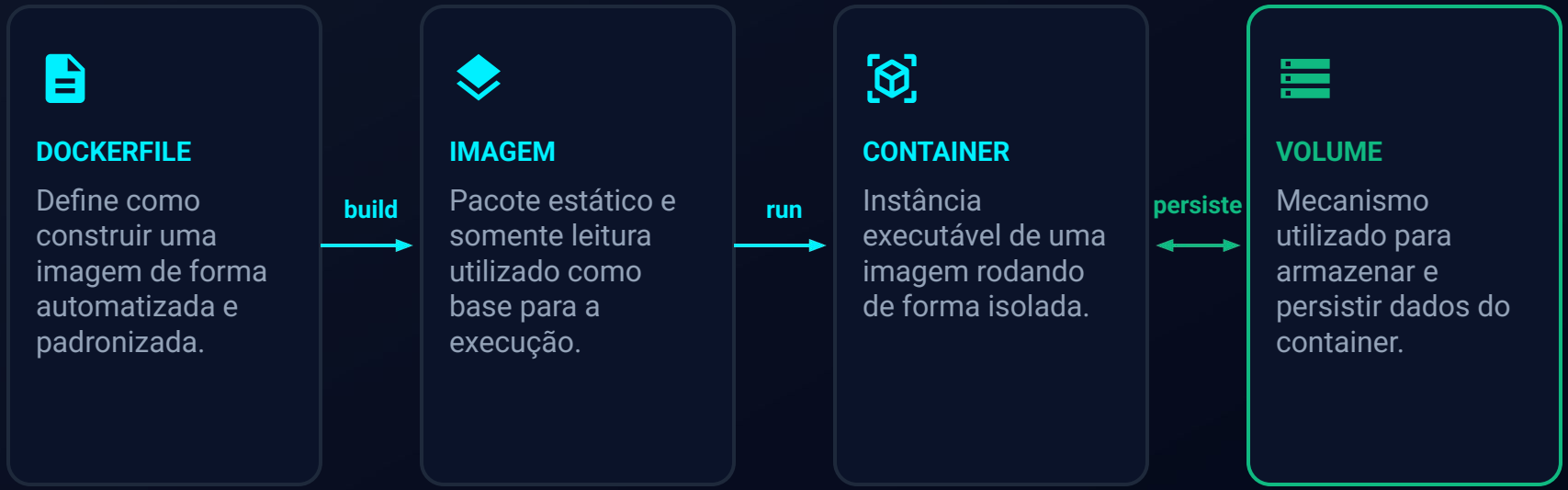
O QUE É DOCKER?

Docker é uma plataforma de código aberto para criar, distribuir e executar containers de forma automatizada.

COMPARAÇÃO ESTRUTURAL

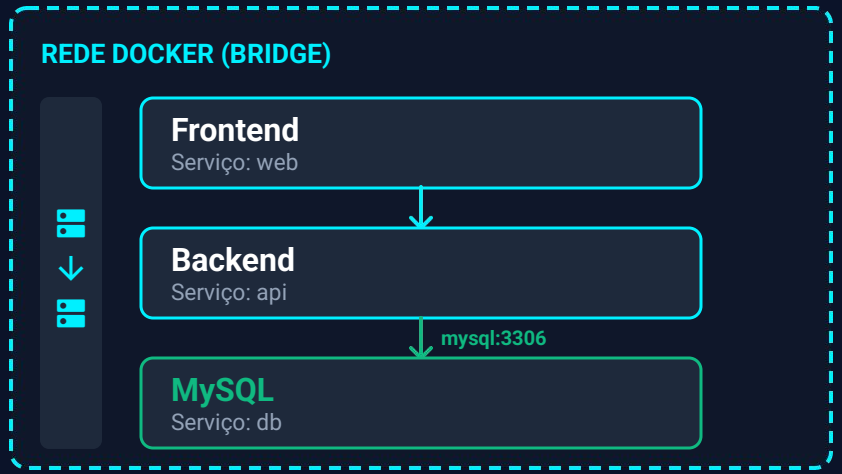
MÁQUINA VIRTUAL	CONTAINER
Aplicação	Aplicação
SO Completo	Container
Hypervisor	Kernel do Host

Conceitos Principais do Docker



Comunicação entre Containers

ESTRUTURA DE REDE



CONECTIVIDADE DIRETA

Comunicação por Nome de Serviço

Containers conectados à mesma rede Docker se comunicam diretamente utilizando o **nome do serviço** como host, dispensando a exposição de portas para a internet.

Exemplo: api → mysql:3306

ATENÇÃO COM O LOCALHOST

localhost = O próprio container

localhost → Próprio Container
mysql:3306 → Serviço MySQL na rede Docker

Exposição de Serviços

MAPEAMENTO DE PORTAS

```
ports: - "5000:5000"
```



ports → **Publica** a porta do container de forma pública no host (acessível externamente).

EXPOSE → **Apenas documenta** a porta interna que a aplicação utiliza para comunicação.

SEGURANÇA POR ISOLAMENTO

Nem todo container precisa ser exposto.

Bancos de dados e microsserviços internos devem permanecer inacessíveis para o mundo externo, operando de forma 100% segura apenas pela rede do Docker.

FLUXO DE ACESSO PROTEGIDO



Persistência de Dados com Volumes

FLUXO DE PERSISTÊNCIA



Isolamento Garantido:

O sistema de arquivos do banco de dados aponta diretamente para uma área segura administrada pelo Docker host.

CICLO DE VIDA DA INFORMAÇÃO



PREMISSAS DE INFRAESTRUTURA

"O container pode ser descartável; os dados não. "

- **Separação Completa**
  Os dados sobrevivem à recriação e deleção das instâncias de serviço.
- **Essencial para DBs**
  Garante persistência de estado em ambientes de microsserviços.

Docker Compose

Ferramenta para definir e executar aplicações multi-container. Através de um arquivo declarativo, configura serviços, redes e volumes de forma coordenada, permitindo que toda a arquitetura suba com um único comando.

```yaml
services:
  postgres:
    container_name: postgres_container
    image: postgres
    environment:
      POSTGRES_USER: ${POSTGRES_USER:-postgres}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-changeme}
      PGDATA: /data/postgres
    volumes:
       - postgres:/data/postgres
    ports:
      - "5432:5432"
    networks:
      - postgres
    restart: unless-stopped
```

Boas Práticas com Docker Compose

IMAGENS

- Use imagens **oficiais ou confiáveis** como base
- Prefira imagens **pequenas e otimizadas** (como Alpine)
- Defina **versões específicas** (evite usar sempre "latest")

CONTAINERS

- **Separe responsabilidades**: um serviço principal por container
- Projete containers para serem **descartáveis e facilmente recriáveis**

DADOS

- Nunca salve dados sensíveis ou persistentes dentro do container
- Utilize **volumes declarados** para garantir a persistência segura

CONFIGURAÇÃO

- Gerencie parâmetros por **variáveis de ambiente** (arquivo .env)
- **Não armazene senhas**, chaves de API ou segredos diretamente no Dockerfile ou controle de versão

REDE

- **Exponha publicamente apenas** as portas de serviços que realmente precisam de acesso externo
- Utilize as **redes internas isoladas** do Docker para a comunicação segura entre containers